Project 2
Implementation of an LALR(1) Parser
Due on Friday, November 22, 2024

Note: You need to COMPLETELY follow the instructions in this sheet.

1. **Problem:**

In this project you are requested to use the tool **Bison** to generate an LALR parser for the small language **Cactus**. The context-free grammar for the small language **Cactus** is as follows:

program → **Program Begin** declarations statements **End**
declarations → declarations **Var Identifier** | ε
statements → statements statement | ε
statement → **Set Identifier =** arithExpr
| **If** boolExpr **Then** statements **EndIf**
| **If** boolExpr **Then** statements **Else** statements **EndIf**
| **While** boolExpr **Do** statements **EndWhile**
| **Read Identifier**
| **Write** arithExpr
| **Exit**
boolExpr → boolExpr **Or** boolTerm
| boolTerm
boolTerm → boolTerm **And** boolFactor
| boolFactor
boolFactor → **Not** boolFactor | relExpr
relExpr → arithExpr **==** arithExpr
| arithExpr **<>** arithExpr
| arithExpr **>** arithExpr
| arithExpr **>=** arithExpr
| arithExpr **<** arithExpr
| arithExpr **<=** arithExpr
arithExpr → arithExpr **+** arithTerm
| arithExpr **–** arithTerm
| arithTerm
arithTerm → arithTerm ***** arithFactor
| arithTerm **/** arithFactor
| arithTerm **%** arithFactor
| arithFactor
arithFactor → **-** arithFactor | primExpr

primExpr → **IntConst** | **Identifier** | **(** arithExpr **)**

The parser needs to parse a program written in **Cactus** defined above. The parser should be able to trace the parsing process by using the option "-p" to print each production reduced by the parser. No syntax error recovery is needed. Each syntax error message should include the line number where the error is detected. The format of the error message is as follows:

Syntax error: line xx

The parser reads input from stdin, writes output to stdout, and writes errors to stderr.

If the Cactus program test1 from stdin is given as follows:

```
// A program to sum 1 to n
Program
Begin
    Var   n
    Var   s

    Read n
    If n < 0 Then
        Write -1
        Exit
    Else
        Set s = 0
    EndIf
    While n > 0 Do          算sigma n
        Set s = s + n
        Set n = n – 1
    EndWhile
    Write s
End
```

the output to stdout using –p option should be as follows:

declarations -> <empty>
declarations -> declarations **Var Identifier**
declarations -> declarations **Var Identifier**
statements -> <empty>
statement -> **Read Identifier**
statements -> statements statement

primExpr -> **Identifier**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithTerm
primExpr -> **IntConst**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithTerm
relExpr -> arithExpr < arithExpr
boolFactor -> relExpr
boolTerm -> boolFactor
boolExpr -> boolTerm
statements -> <empty>
primExpr -> **IntConst**
arithFactor -> primExpr
arithFactor -> **-** arithFactor
arithTerm -> arithFactor
arithExpr -> arithTerm
statement -> **Write** arithExpr
statements -> statements statement
statement -> **Exit**
statements -> statements statement
statements -> <empty>
primExpr -> **IntConst**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithTerm
statement -> **Set Identifier =** arithExpr
statements -> statements statement
statement -> **If** boolExpr **Then** statements **Else** statements **EndIf**
statements -> statements statement
primExpr -> **Identifier**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithTerm
primExpr -> **IntConst**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithTerm

relExpr -> arithExpr > arithExpr
boolFactor -> relExpr
boolTerm -> boolFactor
boolExpr -> boolTerm
statements -> <empty>
primExpr -> **Identifier**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithTerm
primExpr -> **Identifier**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithExpr **+** arithTerm
statement -> **Set Identifier =** arithExpr
statements -> statements statement
primExpr -> **Identifier**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithTerm
primExpr -> **IntConst**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithExpr **–** arithTerm
statement -> **Set Identifier =** arithExpr
statements -> statements statement
statement -> **While** boolExpr **Do** statements **EndWhile**
statements -> statements statement
primExpr -> **Identifier**
arithFactor -> primExpr
arithTerm -> arithFactor
arithExpr -> arithTerm
statement -> **Write** arithExpr
statements -> statements statement
program -> **Program Begin** declarations statements **End**

2. Handing in your program

The executable file should be named “parser.” In Windows,

```
flex cactus.l
bison -d cactus.y
```

```
gcc -o parser.exe lex.yy.c cactus.tab.c
type test1.txt | .\parser.exe
```

In Ubuntu,

```
flex cactus.l
bison -d cactus.y
gcc -o parser lex.yy.c cactus.tab.c
parser < test1
```

To turn in this project, upload a flex file cactus.l and a bison file cactus.y to eCourse2 site.

3. Grading

The grading is based on the correctness of your program. The correctness will be tested by a number of test cases designed by the instructor and teaching assistants.

It is best to incrementally build your program so that you always have a partially-correct working program. It is also best to construct a shell script to systematically test your program.